

Anatomia del codice AR

Ogni riga dei cinque file `index.html` spiegata: cosa fa, perché c'è, e se serve davvero. Con la lista di cosa togliere e cosa lasciare prima della consegna.

01 Come funziona, in breve

03 Lo scheletro HTML

05 Gli asset e il Draco

07 Le luci

09 sharpen-textures

11 Le tre varianti

13 Prima della consegna

02 Le due librerie

04 a-scene e i parametri AR.js

06 Il marker e l'oggetto

08 force-doubleside

10 env-lighting

12 I cinque file a confronto

14 Glossario e fonti

01 Come funziona, in breve

Prima di entrare nel codice, la catena completa. Questa è una sequenza reale: ogni passo dipende dal precedente.

1. Il browser apre la pagina e scarica le due librerie: **A-Frame** (il motore 3D) e **AR.js** (il riconoscimento del marker).
2. AR.js chiede l'accesso alla fotocamera e mostra il video della webcam come sfondo della pagina.
3. Per ogni fotogramma del video, AR.js cerca il disegno del marker stampato. Quando lo trova, calcola dove si trova nello spazio rispetto alla fotocamera: distanza, inclinazione, rotazione.
4. A-Frame usa quei numeri per posizionare il modello 3D esattamente sopra il marker, e lo disegna sopra al video.
5. Il modello viene illuminato dalle luci che hai definito nella scena — non dalla luce vera della stanza. Questo è il punto chiave che spiega quasi tutti i problemi di resa che abbiamo incontrato.

Il malinteso più comune

La fotocamera riprende la stanza, ma quella luce reale non tocca il modello 3D. Il modello è illuminato solo dalle righe `<a-entity light=...>` che scrivi tu. Per questo un oggetto può sembrare “troppo scuro” o “abbagliato” rispetto all'ambiente attorno: sono due mondi luminosi separati, sovrapposti nella stessa immagine.

02 Le due librerie

```
<script src="https://aframe.io/releases/1.3.0/aframe.min.js"></script>
<script src="https://cdn.jsdelivr.net/gh/AR-js-org/AR.js@3.4.8/aframe/build/aframe-ar.js"></script>
```

aframe.min.js – versione 1.3.0

A-Frame, il framework che permette di scrivere scene 3D con tag HTML (`<a-scene>`, `<a-entity>`). Sotto il cofano usa **three.js**, la libreria WebGL vera e propria: ogni volta che nel codice compare `THREE.` qualcosa, stai parlando direttamente con three.js. `.min` significa “minificata”, cioè compressa togliendo spazi e a capo per scaricarla più in fretta.

aframe-ar.js – AR.js 3.4.8

Aggiunge ad A-Frame i tag e i componenti per la realtà aumentata: `<a-marker>` e l'attributo `arjs`. È la parte che analizza il video della webcam e riconosce il marker.

L'ordine dei due script

Non è scambiabile. AR.js si registra dentro A-Frame come componente, quindi A-Frame deve già esistere quando AR.js viene caricato. Invertirli rompe tutto.

Attenzione pratica per il giorno della discussione

Entrambe le librerie vengono scaricate da internet ogni volta che si apre la pagina. Senza connessione — o con il wifi dell'Accademia lento — la pagina si apre bianca e l'AR non parte. Se vuoi essere al sicuro, si possono scaricare i due file e metterli nella cartella del progetto, cambiando `src` in `src="aframe.min.js"` e `src="aframe-ar.js"`.

03 Lo scheletro HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Tesi AR - Motorola DynaTAC</title>
</head>
<body style="margin: 0px; overflow: hidden;">
```

<!DOCTYPE html>

Dice al browser: “questa pagina segue lo standard HTML moderno”. Senza, il browser entra in *quirks mode*, una modalità di compatibilità con i siti degli anni '90 in cui alcune regole di layout cambiano. **Nel file del Motorola manca** — è l'unico dei cinque. Va aggiunto come prima riga.

<html>

Il contenitore di tutta la pagina. Sarebbe buona pratica scrivere `<html lang="it">` per dichiarare la lingua: aiuta i lettori di schermo e i motori di ricerca. In nessuno dei tuoi file c'è, ma è una rifinitura, non un errore.

<head> e <title>

`head` contiene le informazioni sulla pagina, non il contenuto visibile. `title` è il nome che appare nella scheda del browser: “Tesi AR - Motorola DynaTAC”.

margin: 0px

I browser mettono di default un margine di circa 8 pixel attorno al contenuto della pagina. Qui lo azzeri, così il video della fotocamera arriva davvero fino ai bordi dello schermo, senza una cornice bianca.

overflow: hidden

Elimina le barre di scorrimento. Il video a tutto schermo tende a “sforare” di un paio di pixel e farebbe comparire una barra laterale inutile, con il rischio che il telefono scrolli invece di mostrare l'AR.

04 <a-scene> e i parametri di AR.js

```
<a-scene
  embedded
  arjs="sourceType: webcam; debugUIEnabled: false; smooth: true;
    smoothCount: 10; smoothTolerance: 0.01; patternRatio: 0.9;">
```

<a-scene>

La radice della scena 3D. Tutto ciò che è tridimensionale sta dentro questo tag: modelli, luci, camera. A-Frame crea qui il canvas WebGL su cui viene disegnato tutto.

embedded

Un attributo senza valore (basta che ci sia). Dice ad A-Frame di non prendersi tutto lo schermo con i suoi stili di default e di non mostrare il pulsante “Enter VR”. Serve ad AR.js: il video della fotocamera deve poter stare dietro al canvas, e senza `embedded` il canvas lo coprirebbe.

arjs="..."

La configurazione di AR.js. È una sola stringa con più impostazioni separate da punto e virgola, nella forma `nome: valore;`.

sourceType: webcam

Da dove prendere le immagini da analizzare: la fotocamera del dispositivo. Le alternative sono `image` o `video`, usate per fare test su un file registrato invece che dal vivo.

debugUIEnabled: false

Mostra o nasconde il *pannello di debug* di AR.js: un riquadro sovrapposto con i dati tecnici del tracciamento (fotogrammi al secondo, se il marker è visto, i parametri interni). `false` lo nasconde.

smooth: true

Attiva il *livellamento* della posizione. Senza, il modello trema e vibra a ogni fotogramma, perché il riconoscimento del marker ha sempre un piccolo errore che cambia di continuo.

smoothCount: 10

Su quanti fotogrammi fare la media per calcolare la posizione. Più alto = più stabile ma più “lento” a seguire i movimenti veloci; più basso = più reattivo ma più tremolante. 10 è un buon compromesso.

smoothTolerance: 0.01

Sotto questa soglia, uno spostamento viene considerato rumore e ignorato. Impedisce al modello di vibrare quando il telefono è fermo ma la mano trema un po'.

patternRatio: 0.9

Il rapporto fra l'immagine interna del marker e il marker completo compresa la cornice nera. Il valore di default di AR.js è 0.5 (cornice spessa, immagine piccola). 0.9 significa cornice sottile e immagine grande. **Deve corrispondere esattamente al valore usato quando hai generato il file `.patt` e stampato il marker**, altrimenti il riconoscimento non funziona o è instabile.

La tua domanda: false significa che non funziona?

No, e va lasciato esattamente così. `debugUIEnabled` non accende o spegne l'AR: accende o spegne *il pannello di diagnostica sovrapposto al video*. `false` = pannello nascosto, cioè l'aspetto pulito che vuoi per un'installazione. Se lo mettesti `true`, comparirebbe un riquadro con numeri e scritte tecniche sopra al tuo oggetto — utile mentre si sviluppa, brutto in mostra. Quindi: `false` è la scelta giusta per la consegna, in tutti e cinque i file.

05 Gli asset e la compressione Draco

```
<a-scene gltf-model="dracoDecoderPath: https://www.gstatic.com/draco/versioned/decoders/1.5.6/" ...>

  <a-assets>
    <a-asset-item id="nextcube" src="NeXT_Cube.glb?v=2"></a-asset-item>
  </a-assets>
```

<a-assets>

Il sistema di precaricamento di A-Frame. Tutto ciò che sta qui dentro viene scaricato *prima* che la scena parta: così il modello non appare a scatti o a pezzi mentre si carica. La scena aspetta che gli asset siano pronti prima di dire “sono carica”.

<a-asset-item id="nextcube" src="...">

Dichiara un file da precaricare. `id` è il nome con cui lo richiamerai: più sotto scriverai `gltf-model="#nextcube"`, e il cancelletto significa “l'elemento che ha quell'id”. È lo stesso meccanismo dei link interni in HTML.

?v=2 (solo nel NeXT Cube)

Non fa parte del nome del file: è una *query string*, un pezzetto aggiunto dopo il punto interrogativo. Il browser la considera un indirizzo diverso, quindi è costretto a riscaricare il file invece di usare la copia vecchia che ha in memoria. È il trucco standard per aggirare la cache quando sostituisci un `.glb` mantenendo lo stesso nome — esattamente il problema che ci ha fatto impazzire più volte. Puoi lasciarlo, o alzarlo a `?v=3` ogni volta che ricarichi un modello aggiornato.

dracoDecoderPath

Draco è un algoritmo di compressione per geometrie 3D sviluppato da Google: riduce un `.glb` anche di 3–4 volte. Ma il browser da solo non sa decomprimerlo: serve un piccolo programma aggiuntivo, il *decoder*. Questo parametro dice ad A-Frame dove scaricarlo — in questo caso dai server di Google (`gstatic.com`).

Perché nel Motorola non c'è

Perché quel modello non è compresso con Draco. Se lo fosse, senza questa riga non si caricherebbe affatto. È la prova che quel `.glb` è salvato non compresso — probabilmente perché è già piccolo di suo.

Nota

Anche il decoder Draco viene scaricato da internet. Come per le librerie, se vuoi una versione a prova di wifi va scaricato e messo in una cartella locale del progetto.

06 Il marker e l'oggetto

```
<a-marker type="pattern" url="pattern-ouraring.patt">
  <a-entity id="model-oura" force-doubleside gltf-model="#oura"
    scale="40 40 40" position="-0.006 0 0" rotation="0 0 0"></a-entity>
</a-marker>
```

<a-marker>

Rappresenta il marker fisico stampato. Tutto ciò che scrivi *dentro* questo tag è visibile solo quando la fotocamera inquadra quel marker, e si muove insieme a lui. Appena il marker esce dall'inquadratura, l'oggetto sparisce.

type="pattern"

Il tipo di marker: un disegno personalizzato tuo. Le alternative sono `barcode` (codici numerici generati) e `preset` (i marker standard “hiro” e “kanji” che trovi in tutti i tutorial). `pattern` è la scelta giusta per una tesi, perché ti permette marker con la tua grafica.

url="pattern-ouraring.patt"

Il file `.patt` è la “firma” del tuo marker: un file di testo con i valori dei pixel dell'immagine, che AR.js confronta con quello che vede nel video. Si genera con il marker training tool di AR.js partendo dall'immagine stampata.

<a-entity>

L’“entità”: un oggetto generico della scena, che diventa qualcosa di specifico in base ai componenti che gli attacchi. Qui gli attacchi `gltf-model` e diventa un modello 3D.

id="model-oura"

Solo un'etichetta. In questi file non è richiamata da nessuna parte (il `#oura` del modello si riferisce all'asset, non a questa entità). Non fa male e rende il codice più leggibile — puoi tenerla.

gltf-model="#oura"

Il componente che carica e mostra il modello 3D, prendendolo dall'asset con quell'id. **glTF** è il formato standard (lo “JPEG del 3D”), **.glb** è la sua versione binaria: un unico file che contiene geometria, materiali e texture insieme.

scale="40 40 40"

Moltiplicatore di dimensione sui tre assi X, Y, Z. Tre numeri uguali = ingrandimento uniforme. Il valore non ha un significato assoluto: dipende da quanto era grande il modello quando l'hai esportato da Blender. L'anello è modellato in scala reale (2 cm) e il marker è grande circa come un foglio, quindi serve moltiplicare per 40; il Motorola era già grande e gli basta 2.

position="-0.006 0 0"

Spostamento rispetto al centro del marker, in metri, sugli assi X (destra-sinistra), Y (in alto, perpendicolare al foglio) e Z (avanti-indietro sul piano del foglio). Serve a correggere i modelli il cui “punto zero” in Blender non coincide con il centro dell'oggetto. Il NeXT Cube ha **-0.366 0.004 -0.9**: uno scostamento grande, segno che quel modello ha l'origine molto spostata rispetto alla geometria.

rotation="0 -90 0"

Rotazione in **gradi** attorno ai tre assi. Nel Motorola il **-90** sull'asse Y è un quarto di giro in orizzontale: serviva perché il modello usciva da Blender girato di lato rispetto al marker. Ruotare qui è più comodo che rifare l'export.

force-doubleside e sharpen-textures

Attributi senza valore: sono i componenti personalizzati definiti nel **<script>** in cima al file. Scriverli qui è ciò che li “accende” su questo oggetto. Vedi le sezioni 08 e 09.

07 Le luci

```
<a-entity light="type: ambient; color: #FFF; intensity: 1.3"></a-entity>
<a-entity light="type: directional; color: #FFF; intensity: 1.0" position="0.5 1 2"></a-entity>
<a-entity light="type: directional; color: #FFF; intensity: 0.5" position="-1 0.5 1"></a-entity>

<a-entity camera></a-entity>
```

type: ambient

Luce ambientale: illumina tutto in modo uniforme, da ogni direzione, senza produrre ombre né differenza fra le facce. Da sola appiattisce l'oggetto (sembra un disegno piatto), ma impedisce che le zone in ombra diventino nere.

type: directional

Luce direzionale: raggi paralleli che arrivano tutti dalla stessa direzione, come il sole. È quella che crea il chiaroscuro e dà volume all'oggetto.

position su una luce direzionale

Attenzione a non fraintenderlo: per una luce direzionale conta **solo la direzione**, non la distanza. `position="0.5 1 2"` significa “la luce arriva da un punto in alto ($Y=1$), leggermente a destra ($X=0.5$) e davanti ($Z=2$), puntata verso il centro della scena”. Raddoppiare i numeri non cambia nulla, cambia solo l'angolo.

intensity

La forza della luce. Non ha un'unità fisica reale: è un moltiplicatore. 1.0 è il valore neutro.

color: #FFF

Colore della luce in notazione esadecimale abbreviata: `#FFF` equivale a `#FFFFFF`, bianco pieno. Una luce colorata tingerebbe l'oggetto.

<a-entity camera>

Il punto di vista della scena. In AR.js, la fotocamera resta ferma e il marker (con l'oggetto sopra) si muove rispetto a lei, seguendo quello che vede la webcam. Senza questo tag non ci sarebbe nessun osservatore e non vedresti nulla.

Perché servono luci esplicite

A-Frame mette due luci di default in ogni scena. Ma appena ne dichiari una tua, quelle di default spariscono: da quel momento l'illuminazione è interamente responsabilità tua. Per questo le tre righe vanno sempre tenute insieme.

Cosa non torna nei tuoi cinque file

Sommando le tre intensità: View-Master **0,66** · Oura Ring **2,8** · Apple Vision Pro **2,8** · Motorola **3,5** · NeXT Cube **8,2**. Fra il View-Master e il NeXT Cube c'è un fattore **12×**. Non è di per sé un errore — ogni modello ha materiali diversi e le luci sono state regolate a occhio su ciascuno — ma è la prima cosa che un relatore attento può chiederti. La risposta onesta e corretta è: “l'intensità è stata calibrata sul singolo modello, perché i materiali hanno colori base e valori di *roughness* molto diversi fra loro”. Se però il View-Master ti sembra scuro in AR, è lì che devi guardare.

08 Il componente force-doubleside

Da qui in poi siamo nel `<script>` dentro `<head>`. Questi blocchi *definiscono* dei comportamenti; si attivano solo quando li richiami come attributo nell'HTML. Sono due pezzi accoppiati: se togli uno dei due senza l'altro, la console dà errore.


```

AFRAME.registerComponent('force-doubleside', {
  init: function () {
    this.el.addEventListener('model-loaded', () => {
      this.el.object3D.traverse((node) => {
        if (node.isMesh && node.material) {
          if (Array.isArray(node.material)) {
            node.material.forEach(m => { m.side = THREE.DoubleSide; });
          } else {
            node.material.side = THREE.DoubleSide;
          }
        }
      });
    });
  }
});

```

AFRAME.registerComponent('nome', { ... })

Il modo ufficiale di A-Frame per creare un comportamento nuovo. Il primo argomento è il nome con cui lo richiamerai nell'HTML, il secondo è un oggetto che ne descrive il funzionamento.

init: function () { ... }

Uno dei “metodi del ciclo di vita” previsti da A-Frame: viene eseguito una volta sola, quando il componente viene attaccato all'elemento. È il posto dove si fa la preparazione iniziale.

this.el

L'elemento HTML a cui il componente è attaccato. Siccome `force-doubleside` è scritto sull'`<a-entity>` del modello, qui `this.el` è quell'entità.

addEventListener('model-loaded', ...)

“Aspetta che succeda una certa cosa, poi esegui questo codice”. L'evento `model-loaded` viene emesso da A-Frame quando il file `.glb` ha finito di caricarsi. È fondamentale aspettarlo: prima di quel momento i materiali non esistono ancora, e il codice non troverebbe niente da modificare.

() => { ... }

Una *arrow function*, la forma breve per scrivere una funzione in JavaScript moderno. Ha una proprietà importante qui: mantiene il significato di `this` del contesto esterno, così `this.el` continua a funzionare anche dentro il gestore dell'evento.

this.el.object3D

Il ponte fra A-Frame e three.js. Ogni entità A-Frame ha dietro un oggetto three.js vero e proprio, e `object3D` è il modo per raggiungerlo e lavorarci direttamente.

```
.traverse((node) => { ... })
```

Percorre tutto l'albero: l'oggetto e tutti i suoi figli, nipoti e così via, eseguendo la funzione su ognuno. Serve perché un `.glb` non è un blocco unico ma una gerarchia di oggetti — nel tuo Apple Vision Pro erano 18 nodi.

```
if (node.isMesh && node.material)
```

Un filtro. Nell'albero ci sono anche nodi che non sono disegnabili (gruppi vuoti, punti di riferimento, luci). `isMesh` seleziona solo quelli che hanno geometria vera, e `&&` (“e”) aggiunge la condizione che abbiano anche un materiale. Senza questo controllo il codice darebbe errore sul primo nodo vuoto.

```
if (Array.isArray(node.material))
```

Una mesh può avere **più materiali contemporaneamente**, uno per ciascun gruppo di facce: in quel caso `node.material` non è un materiale solo ma una lista. È esattamente il caso della tua mesh “fascia retro.1” del Vision Pro, che ha cinque materiali diversi sullo stesso oggetto. Questa riga distingue i due casi: se è una lista li scorre tutti con `forEach`, altrimenti ne modifica uno solo.

```
m.side = THREE.DoubleSide
```

Il cuore del componente. Ogni triangolo di una mesh ha un davanti e un dietro, decisi dalla direzione della sua *normale*. Per risparmiare calcolo, WebGL disegna di default solo il davanti (`THREE.FrontSide`): il dietro viene scartato, ed è il motivo per cui una faccia con la normale invertita risulta invisibile e vedi “attraverso” il modello. `DoubleSide` dice di disegnare entrambi i lati sempre. Esiste anche `BackSide`, che fa l'opposto — lo usiamo nella sezione 10 per il cielo.

Da sapere se te lo chiedono

Questa è una correzione “a valle”: risolve il sintomo lasciando intatto il modello. La soluzione alla radice sarebbe ricalcolare le normali in Blender (*Mesh* → *Normals* → *Recalculate Outside*). Il costo di `DoubleSide` è un po' più di lavoro per la scheda grafica, trascurabile per modelli di queste dimensioni.

09 Il componente sharpen-textures

Presente solo nell'Apple Vision Pro, perché è l'unico modello con texture di tessuto ripetute molte volte.

```

var maxAniso = renderer ? renderer.capabilities.getMaxAnisotropy() : 1;
this.el.object3D.traverse((node) => {
  if (node.isMesh && node.material) {
    var mats = Array.isArray(node.material) ? node.material : [node.material];
    mats.forEach((m) => {
      ['map', 'normalMap', 'roughnessMap', 'metalnessMap', 'alphaMap', 'emissiveMap'].forEach((slot) => {
        if (m[slot]) {
          m[slot].anisotropy = maxAniso;
          m[slot].needsUpdate = true;
        }
      });
    });
  }
});
});

```

Il filtro anisotropico

È il problema che risolve. Quando guardi una superficie texturizzata *di sbieco* — come succede sempre in AR, perché il telefono non è mai perfettamente frontale al marker — la texture si schiaccia e il browser, per evitare l'effetto scalettato, la sfoca. Il filtro anisotropico corregge questo, mantenendo la texture nitida anche a forte angolo. Costa un po' di prestazioni, quindi di default è impostato al minimo (valore 1, cioè disattivato).

renderer.capabilities.getMaxAnisotropy()

Chiede alla scheda grafica qual è il massimo livello che supporta (tipicamente 16). Non si scrive un numero fisso perché il limite varia da dispositivo a dispositivo.

renderer ? ... : 1

L'*operatore ternario*, una scorciatoia per “se... allora... altrimenti”: si legge “se `renderer` esiste usa il suo valore massimo, altrimenti usa 1”. È una rete di sicurezza nel caso il renderer non sia ancora pronto.

['map', 'normalMap', ...]

La lista delle *mappe* di un materiale PBR, cioè delle immagini che ne descrivono le diverse proprietà: `map` è il colore base, `normalMap` simula i rilievi (le coste del tessuto), `roughnessMap` dice dove è lucido e dove opaco, `metalnessMap` dove è metallo, `alphaMap` la trasparenza, `emissiveMap` le parti che emettono luce. Il codice le scorre tutte e applica il filtro a ciascuna, perché anche la mappa dei rilievi sfocata rovina il risultato.

m[slot]

Sintassi con parentesi quadre: invece di scrivere `m.map`, si usa il nome contenuto nella variabile `slot`. È ciò che permette di scorrere la lista invece di ripetere sei volte lo stesso codice.

needsUpdate = true

Avverte three.js che la texture è cambiata e va ricaricata sulla scheda grafica. Senza, la modifica resterebbe scritta in memoria ma non avrebbe alcun effetto visibile: è un errore classico.

10 Il componente env-lighting

È il blocco più lungo e quello che spaventa di più, ma fa una cosa sola: **costruisce uno studio fotografico finto e invisibile, perché i materiali metallici abbiano qualcosa da riflettere.**

Il motivo è nel modello PBR: un materiale con `metallicFactor` alto non ha quasi colore proprio — il suo aspetto è *quasi interamente ciò che riflette*. In Blender non te ne accorgi perché l'anteprima genera da sola un ambiente di default. In una pagina web quell'ambiente non c'è: il metallo riflette il vuoto, e il vuoto è nero. È esattamente il bug che abbiamo trovato sul pezzo del naso del Vision Pro.

Il contenitore

```
var sceneEl = this.el;
var doIt = function () {
  var renderer = sceneEl.renderer;
  // ...
};
if (sceneEl.hasLoaded) { doIt(); } else { sceneEl.addEventListener('loaded', doIt); }
```

var sceneEl = this.el

Qui il componente è scritto sull'`<a-scene>`, non su un oggetto: quindi `this.el` è la scena intera. La variabile serve solo a poterla richiamare comodamente più sotto.

var doIt = function () { ... }

Tutto il lavoro viene messo dentro una funzione con un nome, invece di eseguirlo subito. Così può essere richiamata in due momenti diversi, come fa la riga finale.

if (sceneEl.hasLoaded) { doIt(); } else { ... }

Una doppia sicurezza: “se la scena è già pronta, fallo adesso; se non lo è ancora, mettiti in attesa dell'evento `loaded` e fallo allora”. Serve perché non si può sapere in anticipo se il componente verrà attivato prima o dopo che la scena sia pronta, e in entrambi i casi il codice deve funzionare.

sceneEl.renderer

Il `renderer` WebGL di three.js: l'oggetto che disegna materialmente ogni fotogramma sullo schermo. È il pezzo di più basso livello che tocchiamo in tutto il codice.

Il tone mapping (solo nell'Apple Vision Pro)

```
renderer.toneMapping = THREE.ACESFilmicToneMapping;  
renderer.toneMappingExposure = 1.0;
```

Il problema

Il renderer somma luce ambientale + luci direzionali + riflessi dell'ambiente. Quel totale può superare 1.0, cioè il bianco massimo. Senza tone mapping, tutto ciò che supera 1.0 viene semplicemente *tagliato* a bianco pieno: le zone chiare diventano macchie bianche piatte, senza più sfumature. È il motivo per cui la fascia grigia del Vision Pro si vedeva bianca.

THREE.ACESFilmicToneMapping

Invece di tagliare, applica una curva che *comprime* gradualmente le alte luci, come fa la pellicola fotografica. ACES (Academy Color Encoding System) è uno standard nato nel cinema, non un'invenzione di three.js. Le alternative in three.js sono `LinearToneMapping`, `ReinhardToneMapping`, `CineonToneMapping`; ACES è quella che dà il risultato più naturale.

toneMappingExposure = 1.0

L'esposizione, esattamente come su una macchina fotografica: sotto 1 scurisce tutto, sopra 1 schiarisce. 1.0 è neutro.

Il generatore della mappa d'ambiente

```
var pmremGenerator = new THREE.PMREMGenerator(renderer);  
pmremGenerator.compileEquirectangularShader();  
var envScene = new THREE.Scene();
```

PMREMGenerator

La sigla sta per *Prefiltered Mipmapped Radiance Environment Map*. Prende una scena e la trasforma in una mappa d'ambiente pronta per essere riflessa dai materiali. “Prefiltrata” significa che ne prepara più versioni sfocate a diversi livelli: i materiali lucidi useranno quella nitida, quelli opachi quella sfocata. È ciò che rende credibile la differenza fra cromo e metallo satinato.

compileEquirectangularShader()

Precompila in anticipo uno shader per evitare un micro-scatto al primo utilizzo. Piccola curiosità onesta: serve per il metodo `fromEquirectangular` (quando parti da una foto panoramica), mentre qui usiamo `fromScene`. Quindi in questo caso specifico non fa nulla di utile — ma non fa nemmeno danno, puoi lasciarla.

new THREE.Scene()

Una seconda scena, separata da quella che vedi. Non verrà mai mostrata: serve solo come “soggetto da fotografare” per generare i riflessi.

Il cielo sfumato

```
var skyGeo = new THREE.SphereGeometry(50, 32, 32);
var skyMat = new THREE.ShaderMaterial({
  side: THREE.BackSide,
  uniforms: {
    topColor: { value: new THREE.Color(0xffffffff) },
    bottomColor: { value: new THREE.Color(0x666666) }
  },
  vertexShader: "...", fragmentShader: "..."
});
var sky = new THREE.Mesh(skyGeo, skyMat);
envScene.add(sky);
```

SphereGeometry(50, 32, 32)

Una sfera di raggio 50, divisa in 32 segmenti in orizzontale e 32 in verticale (più segmenti = più liscia, ma più pesante). È la “cupola del cielo” che circonda tutto.

side: THREE.BackSide

Qui serve il contrario di `force-doubleside`: siccome siamo *dentro* la sfera, quello che vediamo è la sua faccia interna. `BackSide` disegna solo quella, che è anche più efficiente che disegnarle entrambe.

ShaderMaterial

Un materiale scritto a mano, con il proprio programma che gira sulla scheda grafica. Si usa quando serve un effetto che i materiali pronti non fanno — in questo caso una sfumatura verticale continua.

uniforms

Le variabili che passi da JavaScript al programma della scheda grafica. “Uniform” perché restano costanti per tutta la superficie disegnata (a differenza dei *varying*, che cambiano punto per punto). Qui passi i due colori della sfumatura.

0xffffffff e 0x666666

Colori esadecimali: `0x` è il prefisso dei numeri esadecimali in JavaScript, poi due cifre per rosso, due per verde, due per blu. `ffffff` è bianco, `666666` un grigio medio-scuro. Risultato: cielo bianco in alto che sfuma in grigio in basso, come una giornata nuvolosa.

new THREE.Mesh(geometria, materiale)

Una mesh è sempre l'unione di due cose: una forma e una superficie. Qui la sfera più il materiale sfumato.

I due shader, riga per riga

Sono scritti in **GLSL**, il linguaggio dei programmi che girano sulla scheda grafica. Ogni materiale ne ha due: il *vertex shader* viene eseguito una volta per ogni vertice, il *fragment shader* una volta per ogni pixel.

```
// VERTEX SHADER
varying vec3 vWorldPosition;
void main() {
    vec4 worldPosition = modelMatrix * vec4(position, 1.0);
    vWorldPosition = worldPosition.xyz;
    gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
}
```

varying vec3 vWorldPosition

Dichiara un valore che viene calcolato per ogni vertice e poi *interpolato* automaticamente sui pixel in mezzo, per essere letto dal fragment shader. **vec3** significa “tre numeri decimali” (qui X, Y, Z).

position

Una variabile che three.js fornisce già pronta: le coordinate del vertice nello spazio locale dell'oggetto.

modelMatrix * vec4(position, 1.0)

Trasforma quelle coordinate locali in coordinate del mondo, applicando posizione, rotazione e scala dell'oggetto. Il quarto numero **1.0** serve alla matematica delle matrici e significa “questo è un punto”, non una direzione.

gl_Position = projectionMatrix * modelViewMatrix * ...

gl_Position è l'unico risultato obbligatorio di un vertex shader: dove finisce quel vertice sullo schermo. Le due matrici applicano nell'ordine la posizione della camera e la prospettiva.

```
// FRAGMENT SHADER
uniform vec3 topColor;
uniform vec3 bottomColor;
varying vec3 vWorldPosition;
void main() {
    float h = normalize(vWorldPosition).y;
    gl_FragColor = vec4(mix(bottomColor, topColor, max(h * 0.5 + 0.5, 0.0)), 1.0);
}
```

normalize(vWorldPosition).y

normalize riduce il vettore a lunghezza 1, mantenendone la direzione: in pratica dà la direzione dal centro verso quel punto. Prendendone la componente **.y** ottieni un numero che vale **+1 in cima** alla sfera, **0 all'orizzonte** e **-1 in fondo**: è l’“altezza” di quel punto nel cielo.

`h * 0.5 + 0.5`

Converte quell'intervallo da $-1 \dots +1$ a $0 \dots 1$, che è quello che serve per mescolare due colori.

`max(..., 0.0)`

Una sicurezza: se il risultato fosse negativo lo riporta a 0, così non si ottengono colori impossibili.

`mix(bottomColor, topColor, t)`

Interpolazione lineare: con `t` = 0 restituisce il primo colore, con `t` = 1 il secondo, in mezzo la miscela proporzionale. È la funzione che crea materialmente la sfumatura.

`gl_FragColor = vec4(..., 1.0)`

Il risultato obbligatorio di un fragment shader: il colore di quel pixel. `vec4` perché sono quattro numeri, rosso verde blu e trasparenza; `1.0` significa completamente opaco.

I tre pannelli: uno studio fotografico in miniatura

```
function addPanel(x, y, z, w, h, color) {
  var mat = new THREE.MeshBasicMaterial({ color: color });
  var geo = new THREE.PlaneGeometry(w, h);
  var mesh = new THREE.Mesh(geo, mat);
  mesh.position.set(x, y, z);
  mesh.lookAt(0, 0, 0);
  envScene.add(mesh);
}
addPanel(8, 10, 8, 10, 10, 0xffffffff);
addPanel(-8, 4, -8, 6, 6, 0xffe0b0);
addPanel(0, -8, 6, 8, 8, 0x333333);
```

`function addPanel(x, y, z, w, h, color)`

Una funzione scritta apposta per non ripetere tre volte lo stesso codice. I parametri sono, nell'ordine: posizione X, Y, Z, poi larghezza, altezza e colore.

`MeshBasicMaterial`

Un materiale che **non viene influenzato dalle luci**: mostra sempre esattamente il colore che gli dai. È proprio quello che serve per un pannello che deve essere una sorgente di luce da riflettere, non riceverla.

`PlaneGeometry(w, h)`

Un semplice rettangolo piatto delle dimensioni date.

`mesh.lookAt(0, 0, 0)`

Ruota il pannello in modo che sia rivolto verso l'origine, cioè verso il punto dove sta l'oggetto. Senza, i pannelli resterebbero orientati tutti nello stesso verso e non illuminerebbero correttamente.

I tre pannelli in concreto

È l'illuminazione classica a tre punti di uno studio fotografico, trasposta in codice: un pannello **bianco grande** (10×10) in alto a destra e davanti fa da luce principale; uno **più piccolo e caldo** (6×6, colore crema) in basso a sinistra e dietro schiarisce le ombre senza spegnerle; uno **grigio scuro** (8×8) sotto, davanti, fa da pavimento e impedisce che l'oggetto sembri sospeso nel nulla, dando riflessi più scuri nella parte inferiore.



0xffffffff – luce principale



0xffe0b0 – schiarita calda



0x333333 – pavimento scuro



0x666666 – base del cielo

La generazione finale

```
var envMap = pmremGenerator.fromScene(envScene, 0.04).texture;
sceneEl.object3D.environment = envMap;
pmremGenerator.dispose();
```

fromScene(envScene, 0.04)

“Fotografa” la scena finta in tutte le direzioni e la trasforma nella mappa d'ambiente prefiltrata. Il secondo numero è una leggera sfocatura applicata al risultato: ammorbidisce i bordi netti dei pannelli, così i riflessi sembrano luci diffuse e non rettangoli.

.texture

Il metodo restituisce un contenitore tecnico; `.texture` ne estrae l'immagine vera e propria, l'unica parte che ci serve.

sceneEl.object3D.environment = envMap

La riga che fa succedere tutto. `scene.environment` è una proprietà di three.js che applica una mappa d'ambiente **a tutti i materiali PBR della scena contemporaneamente**, senza doverli toccare uno per uno. Da qui in poi ogni metallo ha qualcosa da riflettere.

pmremGenerator.dispose()

Libera la memoria della scheda grafica usata dal generatore, che ha finito il suo lavoro. La mappa già creata resta valida. È buona educazione di programmazione: senza, resterebbe occupata memoria inutilmente.

11 Le tre varianti di env-lighting

Questo è il punto su cui ti conviene arrivare preparata, perché è la differenza più visibile fra i tuoi file: **lo stesso componente esiste in tre versioni diverse.**

Versione semplice – NeXT Cube

Usa l'evento `renderstart` invece di `loaded`, e al posto del cielo e dei pannelli mette un solo colore grigio uniforme: `envScene.background = new THREE.Color(0x888888)`.

Risultato: riflessi piatti e uniformi, senza direzione. Va benissimo per un oggetto poco lucido come un computer in plastica opaca, e costa meno.

Versione con studio – Oura Ring e View-Master

Cielo sfumato più i tre pannelli. Riflessi molto più credibili, con una direzione precisa: indispensabile su un anello metallico. Manca però il tone mapping.

Versione completa – Apple Vision Pro

Come sopra, più le due righe di tone mapping ACES. È l'unica delle tre che gestisce correttamente le alte luci, ed è nata proprio dal problema della fascia bianca.

Cosa dire se te lo chiedono

Sono versioni successive dello stesso componente, mantenute diverse perché ogni modello aveva esigenze diverse: il NeXT Cube non ha superfici molto riflettenti e non trae vantaggio dai pannelli direzionali; il Vision Pro, con la fascia in tessuto chiaro, era l'unico a soffrire il taglio delle alte luci. Uniformarle tutte alla versione completa è possibile e non guasterebbe — ma allora andrebbero ricontrollate le luci di ogni scena, perché il tone mapping cambia la resa complessiva.

12 I cinque file a confronto

OGGETTO	DOCTYPE	DRACO	FORCE- DOUBLESIDE	SHARPEN- TEXTURES	ENV- LIGHTING	STONE MAPPING	LUCI (SOMMA)
Motorola DynaTAC	manca	—	—	—	—	—	3,5
Oura Ring	✓	✓	✓	—	studio	—	2,8
NeXT Cube	✓	✓	✓	—	semplice	—	8,2
View-Master	✓	✓	✓	—	studio	—	0,66
Apple Vision Pro	✓	✓	✓	✓	completa	✓	2,8

Manca all'appello il sesto oggetto, il Ledger Nano S Plus: quando mi mandi il suo `index.html` completo la tabella.

13 Prima della consegna

TOGLI

I commenti di lavoro rivolti a una persona: `<!-- ROTAZIONE DA VERIFICARE: parti da questa, se il cubo è storto/sdraiato dimmi come appare... -->` nel NeXT Cube, nel View-Master e nel Vision Pro. Sono appunti di una conversazione, non documentazione: in un file consegnato stonano.

TIENI

`debugUIEnabled: false` in tutti e cinque i file. `false` è la configurazione corretta per la consegna: nasconde il pannello di diagnostica, non disattiva l'AR.

TIENI

I commenti tecnici che spiegano cosa fa un componente. Codice commentato è considerato un pregio, non un difetto: dimostra che sai perché hai scritto quelle righe. Semmai riscrivili con parole tue, così sono davvero tuoi.

AGGIUNGI

`<!DOCTYPE html>` come primissima riga del file del Motorola: è l'unico che non ce l'ha.

VALUTA

`lang="it"` su `<html>` in tutti i file. Rifinitura minima, ma è lo standard.

VALUTA

Le luci del View-Master (somma 0,66) e del NeXT Cube (somma 8,2). Se in AR si vedono bene così, lasciali e sappi spiegare perché differiscono; se il View-Master appare scuro, è lì che intervenire.

VALUTA

Scaricare in locale A-Frame, AR.js e il decoder Draco, per non dipendere dalla connessione il giorno della discussione. È la modifica che più riduce il rischio di una brutta figura tecnica.

TIENI

Tutto il resto. Nessun altro pezzo di codice funzionale va rimosso: ogni componente presente serve a un problema reale di quello specifico modello.

La regola d'oro quando cancelli

Un componente vive in due posti: la *definizione* nello `<script>` dentro `<head>`, e la *chiamata* come attributo nell'HTML (`env-lighting` sull' `<a-scene>`, `force-doubleside` sull' `<a-entity>`). Se cancelli solo uno dei due, la pagina dà errore o il componente semplicemente non fa nulla. Cancellali sempre in coppia — ed è già successo una volta di cancellare per sbaglio due righe di codice vero insieme a un commento: dopo ogni pulizia, riapri la pagina e verifica che l'oggetto compaia ancora.

14 Glossario e fonti

WebGL

La tecnologia che permette al browser di disegnare grafica 3D usando la scheda grafica. Tutto quello che vedi passa da lì.

three.js

La libreria JavaScript che rende WebGL utilizzabile senza scrivere codice grafico di basso livello. A-Frame la usa internamente.

PBR (Physically Based Rendering)

Il modello di materiali usato oggi ovunque: invece di “quanto è lucido”, descrive una superficie con parametri fisici — colore base, quanto è metallo (`metallicFactor`), quanto è ruvida (`roughnessFactor`). È lo standard adottato dal formato glTF.

IBL (Image Based Lighting)

Illuminare una scena con un'immagine dell'ambiente invece che con lampade virtuali. È quello che fa `env-lighting`.

Mesh, vertice, normale

Una mesh è un oggetto 3D fatto di triangoli; i *vertici* sono gli angoli dei triangoli; la *normale* è la freccia perpendicolare che indica da che parte è rivolta una faccia.

Shader

Un piccolo programma eseguito dalla scheda grafica, scritto in GLSL. Il *vertex shader* decide dove finiscono i punti, il *fragment shader* di che colore è ogni pixel.

Documentazione ufficiale

- **A-Frame — Writing a Component** — <https://aframe.io/docs/1.3.0/core/component.html>: il sistema dei componenti, `registerComponent`, `init`, `this.el`.
- **A-Frame — Light component** — <https://aframe.io/docs/1.3.0/components/light.html>: tipi di luce, `intensity`, comportamento delle luci di default.
- **AR.js Documentation** — <https://ar-js-org.github.io/AR.js-Docs/>: parametri di `arjs`, marker pattern, `patternRatio`, generazione dei file `.patt`.

- [three.js — Material.side](https://threejs.org/docs/#api/en/materials/Material.side) — <https://threejs.org/docs/#api/en/materials/Material.side>: `FrontSide`, `BackSide`, `DoubleSide`.
- [three.js — PMREMGGenerator](https://threejs.org/docs/#api/en/extras/PMREMGGenerator) — <https://threejs.org/docs/#api/en/extras/PMREMGGenerator>: generazione delle mappe d'ambiente, `fromScene`.
- [three.js — WebGLRenderer.toneMapping](https://threejs.org/docs/#api/en/renderers/WebGLRenderer.toneMapping) — <https://threejs.org/docs/#api/en/renderers/WebGLRenderer.toneMapping>: i tone mapping disponibili e l'esposizione.
- [three.js — Texture.anisotropy](https://threejs.org/docs/#api/en/textures/Texture.anisotropy) — <https://threejs.org/docs/#api/en/textures/Texture.anisotropy>: filtro anisotropico e `getMaxAnisotropy`.
- [Khronos — glTF 2.0 Specification](https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html) — <https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>: il formato dei modelli e il modello PBR dei materiali.
- [Google — Draco](https://google.github.io/draco/) — <https://google.github.io/draco/>: la compressione delle geometrie 3D e il decoder.

Guida di lettura dei cinque file `index.html` del progetto — Archeologia Post-Digitale. Aggiornata al 11 settembre 2026; manca il sesto oggetto, Ledger Nano S Plus.